Hospital Management System - Dokumentation Teil 1

Gruppe 1

THWS Würzburg Schweinfurt

Joshua Pfennig: 5123097
Tim Knüttel: 5123101
Jasmin Wander: 5123060
David Heppenheimer: 5123026
Tom Knoblach: 5123034

14. April 2026

Inhaltsverzeichnis

1 Szenariobegründung (5123097) 3

2 Architektur und Infrastruktur (5123026) 4
2.1 Wahl des Datenbanksystems 4
2.2 Bereitstellung und Hosting 4
2.3 Aktueller Stand und Tooling 5
2.4 Ausblick 5

3 Datenbeschaffung und Datenaufbereitung (5123034) 6
3.1 Zielsetzung 6
3.2 Datenquelle 6
3.3 Datenaufbereitung 6
3.4 Datenverarbeitung und technologischer Ansatz 7
3.5 Herausforderungen 7

4 Use Cases: Medikamentenverwaltung (5123060) 8

5 Use Cases: Personenverwaltung (5123101) 13

6 Use Cases: Patienten- und Raumverwaltung (5123097) 17

7 Finales Klassendiagramm 21

8 KI Nutzung 22

1 Szenariobegründung (5123097)

Die zunehmende Digitalisierung im Gesundheitswesen bietet immense Chancen für effizientere Prozesse und die Verbesserung der Patientenversorgung. In einem Umfeld, das als kritische Infrastruktur (KRITIS) eingestuft wird [1], steigen jedoch mit der zunehmenden Vernetzung auch die Anforderungen an die Sicherheit und Integrität der zugrunde liegenden Datenbankstrukturen weiter an.
Die steigende Zahl von Cyberangriffen auf Gesundheitseinrichtungen verdeutlicht, dass dringend robuste Sicherheitskonzepte nötig sind, da nicht mehr nur die Verfügbarkeit der Systeme, sondern gezielt die Vertraulichkeit hochsensibler Daten angegriffen wird. Ein aktuelles Beispiel hierfür ist der Vorfall bei den LUP-Kliniken in Mecklenburg-Vorpommern [6]. Nach einem Ransomware-Angriff im Juli 2025 wurden sensible Patientendaten sowie Informationen über interne Abläufe im Darknet veröffentlicht. Der unkontrollierte Abfluss dieser Informationen ermöglicht beispielsweise gezielte Identitätsdiebstahle und Erpressungsversuche auf Basis sensibler Diagnosen.
Solche Vorfälle zeigen deutlich, dass der Schutz vor unbefugtem Datenzugriff keine rein theoretische Anforderung der DSGVO darstellt, sondern essenziell für das Vertrauensverhältnis zwischen Patienten und den Kliniken ist.

Das konzipierte System adressiert diese komplexen Anforderungen durch eine umfangreiche relationale Datenbankstruktur, die drei zentrale Bestandteile des Klinikalltags integriert.
Die Personalverwaltung bildet dabei die organisatorische Grundlage, indem sie die Differenzierung zwischen ärztlichem Dienst und Pflegepersonal sowie deren Zuordnung zu spezifischen Fachabteilungen sicherstellt.
Darauf aufbauend ermöglicht die Patienten- und Raumverwaltung eine präzise Steuerung der stationären Aufnahmen und eine effiziente Belegungsplanung.
Die dritte Säule bildet die Medikamenten- und Diagnosenverwaltung, welche die medizinische Historie der Patienten dokumentiert.

Aufgrund der hohen Sensibilität dieser Informationen liegt ein besonderer Fokus des Systems auf der Gewährleistung der Datensicherheit. Um Szenarien wie den oben genannten Datenleaks proaktiv zu begegnen, werden daher kritischer Datenfelder sicher verschlüsselt. Ziel ist es, ein System zu schaffen, das die Komplexität eines modernen Krankenhauses, wie beispielsweise der Uniklinik Würzburg, abbildet und gleichzeitig durch moderne Sicherheitsstandards die Vertraulichkeit der Patienten- und Personaldaten auf Datenbankebene schützt.

2 Architektur und Infrastruktur (5123026)

2.1 Wahl des Datenbanksystems

Bei der Planung der Architektur für das vorliegende Krankenhausinformationssystem war es zunächst erforderlich, unterschiedliche Datenbankmodelle für unseren hochspezifischen Kontext abzuwägen. Im direkten Vergleich standen dabei NoSQL-Datenbanken (z. B. dokumentenorientierte wie MongoDB) und klassische relationale Datenbanken (RDBMS).

Obwohl NoSQL-Datenbanken durch ihre Flexibilität bei variierenden oder unstrukturierten Daten glänzen, erfordert unser Anwendungsfall zwingend ein striktes Datenmodell. Das zugrundeliegende Domänenmodell beschreibt ein hochgradig und klar vernetztes Regelwerk aus Patienten (*patient*), Diagnosen (*diagnosis*), Raumbuchungen (*bookings*) und differenzierten Medikationsplänen (Verknüpfung von *medication*, *drugs* und *dose*). Eine Fehlzuweisung bei Betten oder eine Inkonsistenz in der Medikamentenvergabe kann im medizinischen Kontext gravierende Konsequenzen haben. Aufgrund dieser essenziellen Anforderungen an Korrektheit und der Vielzahl an strukturierten Fremdschlüssel-Abhängigkeiten haben wir uns für ein **relationales Datenbanksystem (PostgreSQL)** entschieden. Die ACID-Garantien von PostgreSQL sichern die relationale Integrität dieser sensiblen Stammdaten (wie Ärztetypen, Medikamenten-Stock, Raumkapazitäten und Mitarbeiter), während komplexe Auswertungen (z. B. Stationsbelegung über Joins von *department*, *station* und *rooms*) konsistent und performant abgebildet werden können.

2.2 Bereitstellung und Hosting

Neben der Frage des Datenbanktyps mussten wir zudem über das Hosting entscheiden. Anstatt auf etablierte und pflegeleichte externe Cloud-Services (wie AWS RDS oder Azure SQL) zurückzugreifen, haben wir uns ganz bewusst für eine **Self-Hosted-Datenbank** entschieden.

Da es sich hierbei um ein praxisnahes Studien- bzw. Entwicklungsprojekt handelt, steht vor allem ein uneingeschränkter Zugang zu Hardware-Ressourcen im Vordergrund: Für das ausgiebige Testen, das Entwickeln und Optimieren von anspruchsvollen Datenbankabfragen über unsere vielen Tabellen sowie das Experimentieren mit sehr großen, simulierten Datenmengen benötigen wir extrem viel Rechenleistung. Bei externen Cloud-Anbietern würden diese ressourcenintensiven Operationen, Lasttests und das freie „Rumprobieren“ sehr schnell immense und unberechenbare Kosten (Pay-per-Use) durch dau-

erhafte Compute- und Storage-Nutzung verursachen. Der Self-Hosted-Ansatz auf leistungsstarker, eigener Hardware bietet uns stattdessen eine völlig budgetneutrale Spielwiese. Wir können die enormen CPU- und RAM-Ressourcen nach Belieben ausreizen und die PostgreSQL-Konfiguration dediziert auf das Verarbeiten großer Datenmengen abstimmen, ohne an Preismodelle externer Cloud-Anbieter gebunden zu sein.

2.3 Aktueller Stand und Tooling

Konkret läuft die PostgreSQL-Instanz in einem **Docker-Container**[4] auf einer dedizierten virtuellen Maschine (VM). Durch die Containerisierung ist die Datenbank isoliert, reproduzierbar und bei Bedarf schnell neu aufsetzbar.

Für den schnellen, webbasierten Zugriff auf die Datenbank kommt **pgAdmin 4**[8] zum Einsatz. Über das browserbasierte Webpanel lassen sich Tabellen inspizieren, Ad-hoc-Abfragen ausführen und die Datenbankstruktur visuell überblicken – ohne lokal zusätzliche Software installieren zu müssen. Für tiefergehende Entwicklungsarbeit, das Schreiben und Optimieren komplexer SQL-Abfragen sowie das gezielte Experimentieren mit Daten wird darüber hinaus **DataGrip**[5] von JetBrains als vollwertige Datenbank-IDE verwendet. DataGrip bietet u. a. intelligente Autovervollständigung, Query-Analyse und einen komfortablen Schema-Navigator, was das freie Herumprobieren mit unterschiedlichen Abfragen und Datenmengen erheblich beschleunigt.

2.4 Ausblick

In einem nächsten Schritt ist geplant, die Datenbank über ein dediziertes **Backend** anzubinden, welches die Geschäftslogik kapselt und über eine API bereitstellt. Darauf aufbauend soll ein **Frontend** entwickelt werden, das eine benutzerfreundliche Oberfläche zur Interaktion mit dem System bietet. Durch diese vollständige Architektur aus Datenbank, Backend und Frontend wird es möglich sein, tatsächliche Anwendungsdaten realistisch zu simulieren, Abläufe wie Patientenaufnahmen oder Raumbuchungen end-to-end durchzuspielen und die Ergebnisse visuell aufzubereiten.

3 Datenbeschaffung und Datenaufbereitung (5123034)

3.1 Zielsetzung

Ziel dieses Projektteils ist die strukturierte Beschaffung und Aufbereitung von Daten, um die Datenbank sowie das darauf aufbauende System zu testen. Der Fokus liegt dabei insbesondere auf der Transformation von Rohdaten in ein konsistentes und qualitätsgesichertes Format.

3.2 Datenquelle

Als primäre Datenquelle dienen öffentlich zugängliche Datensätze der Plattform Kaggle [7]. Die Plattform bietet eine Vielzahl an Datensammlungen aus unterschiedlichen Anwendungsbereichen und ist daher gut für datengetriebene Projekte geeignet. Die Datensätze liegen in der Regel als JSON- oder CSV-Dateien vor und enthalten bereits strukturierte Informationen. Dennoch existiert kein Datensatz, der direkt auf unser Datenbankschema passt. Zudem handelt es sich häufig um Rohdaten, die Unstimmigkeiten, fehlende Werte oder uneinheitliche Formate enthalten. Ergänzend wurden weitere Datensätze von Mendeley [3] und Hugging Face [2] verwendet, um die Datenbasis zu erweitern.

3.3 Datenaufbereitung

Ein zentraler Schritt nach der Erstellung der Datenbank ist die Befüllung mit geeigneten Daten. Da keine Datensätze existieren, die direkt dem Datenbankschema entsprechen, müssen die vorhandenen Rohdaten entsprechend angepasst werden.

Der erste Schritt besteht in der Bereinigung der Daten. Dabei werden insbesondere fehlende Werte, inkonsistente Schreibweisen sowie weitere Unstimmigkeiten behandelt. Dies umfasst unter anderem das Vereinheitlichen von Duplikaten sowie das Entfernen unvollständiger Datensätze.

Im Anschluss erfolgt die Transformation der bereinigten Daten, um sie an die Anforderungen des Datenbankschemas anzupassen. Hierbei werden Datentypen entsprechend konvertiert und Attribute gegebenenfalls aufgeteilt oder zusammengeführt. Ziel dieses Schrittes ist es, eine einheitliche und logisch strukturierte Datenbasis zu schaffen.

Vor der weiteren Verarbeitung werden die Daten abschließend validiert, um

eine hohe Datenqualität sicherzustellen und Probleme beim Einfügen in die normalisierte Datenbank sowie bei späteren Analysen zu vermeiden.

3.4 Datenverarbeitung und technologischer Ansatz

Für die Umsetzung der Datenverarbeitung und -aufbereitung wurde Python verwendet. Ausschlaggebend hierfür sind insbesondere die umfangreichen Bibliotheken im Bereich der Datenverarbeitung, die einfache Erweiterbarkeit sowie die gute Eignung für automatisierte Prozesse.

Der Import der Daten in die Datenbank erfolgt über automatisierte Python-Skripte. Dabei wird bewusst eine Trennung zwischen Datenaufbereitung und Datenimport eingehalten, um eine klare Modularisierung zu gewährleisten.

Der Importprozess gliedert sich in mehrere Schritte. Zunächst werden die Quelldateien eingelesen. Anschließend erfolgt eine erste Strukturprüfung der Daten, um mögliche Inkonsistenzen frühzeitig zu erkennen. Im letzten Schritt werden die Daten in geeignete Datenstrukturen überführt, sodass sie effizient weiterverarbeitet und in die Datenbank importiert werden können.

Konkrete Entscheidungen bezüglich der eingesetzten Datenbanktechnologien sowie weiterer Tools werden in einem separaten Abschnitt erläutert.

3.5 Herausforderungen

Im Rahmen der Datenaufbereitung traten mehrere Herausforderungen auf. Eine der größten bestand darin, geeignete und realistische Datensätze zu finden. Da der Fokus auf praxisnahen Daten lag, wurden insbesondere Datensätze mit Informationen zu realen Medikamenten oder Krankheiten benötigt.

Solche spezifischen Datensätze sind jedoch nur begrenzt verfügbar, wodurch die Suche zeitaufwendig war und teilweise Kompromisse bei der Auswahl getroffen werden mussten.

Eine weitere Herausforderung bestand darin die Daten so aufzubereiten und so zu verbinden das die Verbindungen zwischen den verschiedenen Tabellen der Datenbank weiterhin sinnhafte Verbindungen darstellen. Ein Beispiel hierfür ist die Verbindung zwischen den Medikamenten und den Dosierungen. Hier musste sichergestellt werden, dass die Angabe der Dosierung auch zu der tatsächlichen Form der Medikamenten passt. So ist es beispielsweise nicht sinnvoll eine Dosierung in Form von Tropfen oder Millilitern anzugeben, wenn es sich bei der Form des Medikaments um eine Tablette handelt.

4 Use Cases: Medikamentenverwaltung (5123060)

Die folgenden Tabellen beschreiben die funktionalen Anforderungen an die Medikamentenverwaltung. Darüber hinaus wird auf den Aufbau der zugehörigen Datenbanktabellen eingegangen.

Titel	**Anlage einer Medikation auf Basis einer Diagnose**
Kontext	Die Erstellung einer Medikation wird durch einen Arzt ausgelöst, nachdem eine Diagnose für einen Patienten gestellt wurde. Voraussetzung ist, dass sowohl die Diagnose im System existiert als auch ein geeignetes Medikament in der Datenbank verfügbar ist.
Beschreibung	Zur Behandlung einer diagnostizierten Erkrankung wird eine Medikation angelegt, indem ein konkretes Medikament ausgewählt und mit einer definierten Dosierung sowie Einnahmefrequenz versehen wird. Zusätzlich wird der Beginn der Behandlung dokumentiert, sodass eine zeitliche Nachvollziehbarkeit gewährleistet ist.
Besonderheiten	Die Medikation muss eindeutig einer bestehenden Diagnose zugeordnet sein. Zudem darf nur auf Medikamente zurückgegriffen werden, die im System als verfügbar geführt werden.

Tabelle 1: Use Case: Medikation anlegen

Titel	**Anpassung und Beendigung einer Medikation**
Kontext	Eine Änderung einer bestehenden Medikation kann während eines laufenden Behandlungsprozesses erforderlich sein, etwa aufgrund veränderter Symptome, unzureichender Wirksamkeit oder neuer medizinischer Erkenntnisse. Alternativ kann eine Medikation beendet werden, wenn die Behandlung abgeschlossen ist oder das Medikament nicht weiter verabreicht werden soll. Voraussetzung ist in beiden Fällen eine bereits bestehende aktive Medikation.
Beschreibung	Eine bestehende Medikation wird hinsichtlich Dosierung, Medikament oder Einnahmefrequenz angepasst bzw. vollständig beendet, indem ein entsprechender Endzeitpunkt dokumentiert wird. Ziel ist es, die Behandlung entweder optimal an den aktuellen Zustand des Patienten anzupassen oder ordnungsgemäß abzuschließen und zeitlich vollständig zu erfassen.
Besonderheiten	Beendete Medikationen dürfen nicht gelöscht werden, da sie weiterhin für die medizinische Historie relevant sind. Inkonsistente Zustände, wie sich widersprechende parallele Medikationen, sind zu vermeiden.

Tabelle 2: Use Case: Medikation ändern bzw. beenden

Titel	**Verwaltung und Verfügbarkeitsprüfung von Arzneimitteln**
Kontext	Dieser Use Case wird durch administratives Personal, die Krankenhausapotheke oder medizinisches Personal ausgelöst. Einerseits erfolgt die Verwaltung von Arzneimitteln bei der Einführung neuer Medikamente oder der Aktualisierung bestehender Einträge. Andererseits wird vor der Verschreibung eines Medikaments dessen Verfügbarkeit geprüft. Voraussetzung sind entsprechende Zugriffsrechte sowie im System erfasste Arzneimittel.
Beschreibung	Arzneimittel werden im System angelegt, bearbeitet oder deaktiviert, wobei Informationen wie Name, Wirkstoff, Typ und Lagerbestand gepflegt werden. Zusätzlich wird im Rahmen der Nutzung eines Medikaments dessen aktueller Bestand überprüft, um sicherzustellen, dass eine ausreichende Menge für die Behandlung vorhanden ist oder gegebenenfalls eine Nachbestellung erforderlich wird.
Besonderheiten	Nur aktive und verfügbare Medikamente dürfen für neue Medikationen verwendet werden. Änderungen am Lagerbestand müssen konsistent gehalten werden, da sie directen Einfluss auf die Verfügbarkeitsprüfung und damit auf medizinische Entscheidungen haben.

Tabelle 3: Use Case: Arzneimittel verwalten und Verfügbarkeit prüfen

Resultierende Entitäten

Basierend auf den beschriebenen Anwendungsfällen wurde ein Datenbankschema entwickelt, das die Prozesse der Medikamentenverwaltung strukturiert abbildet. Im Zentrum steht die Diagnose (`diagnosis`), welche die medizinische Feststellung einer Erkrankung repräsentiert und als Grundlage für therapeutische Maßnahmen dient.

Darauf aufbauend wird die Medikation (`medication`) modelliert, die eine Diagnose mit einem konkreten Arzneimittel verknüpft und durch zeitliche Angaben wie Beginn und Ende der Behandlung ergänzt wird. Dadurch bleibt der Behandlungsverlauf vollständig nachvollziehbar.

Die Entität `drugs` umfasst alle verfügbaren Medikamente sowie deren Eigenschaften wie Name, Wirkstoff, Typ und Lagerbestand. Sie dient als zentrale Stammdatenbasis und ermöglicht gleichzeitig die Prüfung der Verfügbarkeit im Behandlungsprozess.

Die Dosierung (`dose`) konkretisiert die Medikation, indem sie Menge, Einheit und Einnahmefrequenz festlegt. Durch die separate Modellierung dieser Entität wird eine flexible und konsistente Abbildung unterschiedlicher Dosierungsschemata erreicht.

Insgesamt ergibt sich eine strukturierte Abfolge von der Diagnose über die Medikation bis hin zum Arzneimittel und dessen Dosierung, wodurch sowohl die medizinische Logik als auch die Anforderungen an Nachvollziehbarkeit im Datenmodell gewährleistet werden.

Verwendete Enums

Die in der Medikamentenverwaltung verwendeten Enum-Typen dienen der Sicherstellung konsistenter und gültiger Werte innerhalb der Datenbank. Der Enum `drugs_type` definiert die Darreichungsform eines Medikaments (z. B. Tablet, Kapsel oder Injektion), während `dose_unit` die Maßeinheit der Dosierung (z. B. mg, ml oder Einheit) festlegt. Der Enum `dose_frequency` beschreibt schließlich die Einnahmefrequenz eines Medikaments (z. B. mehrmals täglich oder in festen Zeitintervallen). Durch die Verwendung dieser vordefinierten Wertemengen wird die Datenintegrität erhöht und eine einheitliche Verarbeitung im System gewährleistet.

Darüber hinaus werden die Daten in der Tabelle `diagnosis` verschlüsselt gespeichert, um ein hohes Maß an Datensicherheit und Privatsphäre zu gewährleisten.

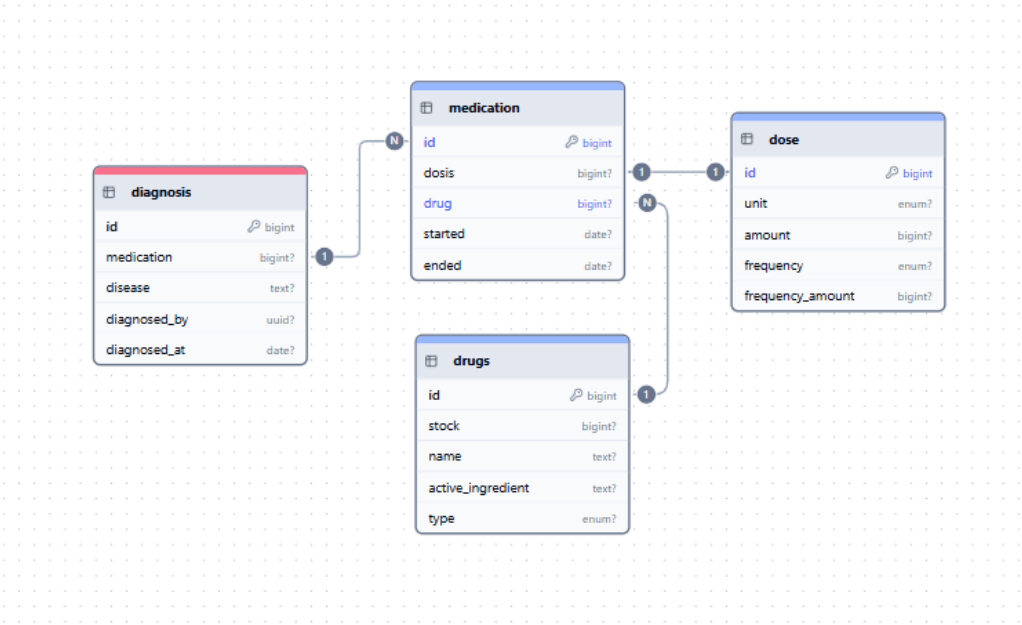

Abbildung 1: Resultierende Klassen zur Medikamentenverwaltung

5 Use Cases: Personenverwaltung (5123101)

Im folgenden werden die Use Cases zur Personenverwaltung beschrieben. Darüber hinaus wird auf den Aufbau der zugehörigen Datenbanktabellen eingegangen.

Titel	**Patientenverwaltung**
Kontext	Damit ein Patient stationär aufgenommen werden kann, muss dieser zuerst als Person in der Krankenhaus Datenbank angelegt werden und eine Zuordnung als Patient erfolgen. Hierbei übernimmt eine Pflegekraft oder Arzt die Daten in das System
Beschreibung	Der Use Case beschreibt die Erfassung und Verwaltung aller Patienten. Bei der Aufnahme werden zunächst die grundlegenden persönlichen Daten erfasst. Anschließend wird die Person als Patient im System geführt, wodurch sie eindeutig als solcher zugeordnet ist. Bestehende Angaben können jederzeit aktualisiert und gepflegt werden

Tabelle 4: Use Case: Patientenverwaltung

Titel	**Mitarbeiterverwaltung**
Kontext	Wenn ein neuer Mitarbeiter ins Krankenhaus kommt oder sich Daten von bestehenden Mitarbeitern ändern erfolgt die Pflege der Daten durch die Administration
Beschreibung	Der Use Case beschreibt die Erfassung und Verwaltung aller Mitarbeitenden im Krankenhaus. Zunächst werden die grundlegenden persönlichen Daten aufgenommen. Anschließend wird die Person als Mitarbeitender im System geführt und einem Fachbereich zugeordnet. Im nächsten Schritt erfolgt die Einteilung entweder als Arzt oder als Pflegekraft. Ärzte erhalten dabei zusätzlich eine genauere fachliche Einordnung sowie eine dienstliche Telefonnummer. Pflegekräfte werden hingegen der Station zugewiesen, auf der sie tätig sind

Tabelle 5: Use Case: Mitarbeiterverwaltung

Datenverwaltung

Bei der Datenverwaltung besteht eine zentrale Besonderheit darin, dass sämtliche personenbezogenen Informationen gebündelt in der Tabelle `person` gespeichert werden. Die Tabellen `patient` und `employee` enthalten hingegen ausschließlich Referenzen auf die entsprechenden Einträge in `person`. Dadurch wird vermieden, dass identische Daten mehrfach gespeichert werden müssen.

Um dennoch eine eindeutige Zuordnung sicherzustellen, besteht sowohl zwischen `person` und `patient` als auch zwischen `person` und `employee` jeweils eine 1:1-Beziehung. Zusätzlich zum Eintrag in der Tabelle `employee` wird für jede Person entweder ein Eintrag in `doctors` oder `nurses` angelegt. Diese Einträge referenzieren jeweils den zugehörigen `employee`-Datensatz und ermöglichen so eine eindeutige fachliche Zuordnung. Auch hier bestehen jeweils 1:1-Beziehungen.

Der Enum `doctors_type` definiert die möglichen Rollen von Ärzten innerhalb des Krankenhauses und bildet damit die hierarchische Struktur des medizinischen Personals ab.

Typ	**Beschreibung**
assistant_physician	Assistenzarzt
resident	Arzt in fachärztlicher Ausbildung
attending_physician	Facharzt mit eigenverantwortlicher Behandlung von Patienten
senior_physician	Oberarzt mit zusätzlicher fachlicher und organisatorischer Verantwortung
consultant	Facharzt mit beratender Funktion, häufig spezialisiert
chief_physician	Chefarzt mit leitender Funktion
head_of_department	Leiter einer Abteilung, verantwortlich für Organisation und Personal

Tabelle 6: Ärztliche Rollen und Hierarchiestufen

Die Verwendung eines Enum-Typs stellt sicher, dass ausschließlich vordefinierte und gültige Rollen in der Datenbank gespeichert werden können. Dies erhöht die Datenintegrität und vereinfacht zugleich die Verarbeitung sowie Auswertung der Daten.

Darüber hinaus werden die personenbezogenen Daten in der Tabelle `person` verschlüsselt gespeichert, um ein hohes Maß an Datensicherheit zu gewährleisten.

Im Folgenden veranschaulicht eine Grafik den strukturellen Aufbau der benötigten Tabellen.

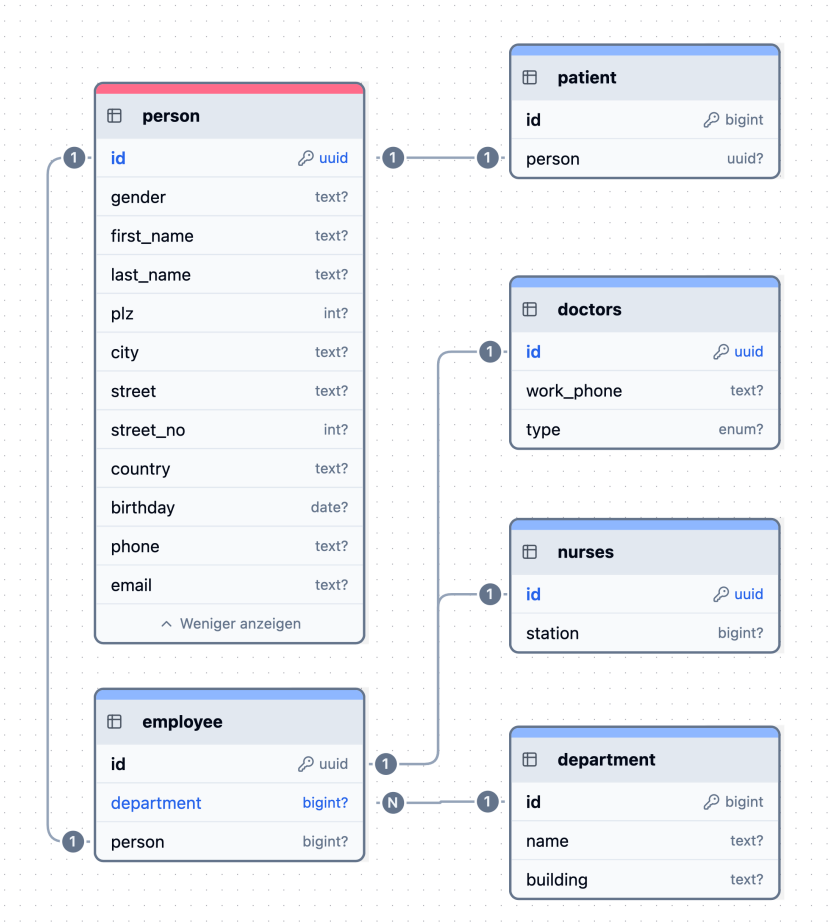

Abbildung 2: Personenverwaltung

6 Use Cases: Patienten- und Raumverwaltung (5123097)

Die folgenden Tabellen beschreiben die funktionalen Anforderungen an die Patienten- und Raumverwaltung, um die entsprechenden Integritätsbedingungen des Datenmodells aufzuzeigen.

Titel	**Stationäre Aufnahme eines Patienten und initiale Zimmerzuweisung**
Kontext	Sobald ein Patient stationär aufgenommen werden soll, etwa nach einer ärztlichen Einweisung oder aus der Notaufnahme heraus, wird dies durch administratives Personal initiiert. Als Voraussetzung muss der Patient bereits im Krankenhausinformationssystem erfasst sein und aktuelle Informationen zur Zimmerbelegung müssen vorliegen, sodass eine geeignete Zuweisung erfolgen kann.
Beschreibung	Zur stationären Aufnahme eines Patienten wird ein bereits im System erfasster Patient einem konkreten Zimmer zugewiesen, wodurch ein aktiver Aufenthalt erzeugt wird. Die Zuweisung erfolgt unter Berücksichtigung der aktuellen Zimmerauslastung sowie organisatorischer Kriterien wie Station oder Fachabteilung. Dokumentiert wird hierbei zudem der Beginn des Aufenthalts.
Besonderheiten	Das zugewiesene Zimmer darf noch nicht vollständig belegt sein, eine Überschreitung der Kapazität ist dementsprechend nicht möglich.

Tabelle 7: Use Case: Stationäre Aufnahme eines Patienten

Titel	**Interne Verlegung und Entlassung eines Patienten**
Kontext	Eine Änderung der aktuellen Zimmerbelegung kann beispielsweise durch medizinische Anforderungen, organisatorische Umstrukturierungen oder eine optimierte Auslastung der Stationen erforderlich sein. Alternativ kann der stationäre Behandlungsaufenthalt auch abgeschlossen sein. Voraussetzung ist in beiden Fällen ein bereits bestehender stationärer Aufenthalt des Patienten sowie für eine interne Verlegung die Verfügbarkeit eines geeigneten Zielzimmers.
Beschreibung	Der Patient wird aus seinem aktuell zugewiesenen Zimmer in ein anderes Zimmer innerhalb des Krankenhauses verlegt bzw. der aktive stationäre Aufenthalt des Patienten wird beendet, indem der zugehörige Buchungseintrag abgeschlossen wird und die Entlassung mit dem entsprechenden Zeitstempel dokumentiert wird. In beiden Fällen wird das zuvor belegte Zimmer wieder für zukünftige Patienten frei.
Besonderheiten	Der bestehende Buchungseintrag wird zur besseren Nachvollziehbarkeit mit dem entsprechenden Zeitstempel geschlossen und ein neuer Eintrag angelegt. Zudem gelten weiterhin die Beschränkungen der Kapazität auch für das neu zugewiesene Zimmer.

Tabelle 8: Use Case: Zimmerverlegung

Resultierende Entitäten

Basierend auf den beschriebenen Anwendungsfällen wurde ein Datenbankschema entwickelt, das die organisatorischen und prozessualen Abläufe präzise abbildet. Die strukturelle Basis bildet dabei die Abteilung (`Department`) als oberste organisatorische Einheit. Sie dient dazu, medizinische Kompetenzen zu bündeln und die zugehörigen Stationen zu gruppieren. Jede Station ist funktional fest einer Abteilung zugeordnet und fungiert als pflegerische Einheit, die wiederum eine Menge von Zimmern (Rooms) umfasst. Diese Zimmer lassen sich über eine eindeutige Zuordnung zur jeweiligen Station lokalisieren und verwalten.

Im Mittelpunkt des Systems steht der Patient als zentrale Entität für sämtliche medizinischen und administrativen Prozesse. Die operative Verbindung zwischen der räumlichen Infrastruktur und den Patienten wird dementsprechend über die Buchungen (Bookings) realisiert. Diese Entität löst die n:m-Beziehung zwischen Patienten und Zimmern auf, indem sie die zeitliche Dimension der Raumbelegung über Start- und Endzeitpunkte abbildet. Somit wird jederzeit nachvollziehbar, welcher Patient in welchem Zeitraum in welchem Raum untergebracht ist.

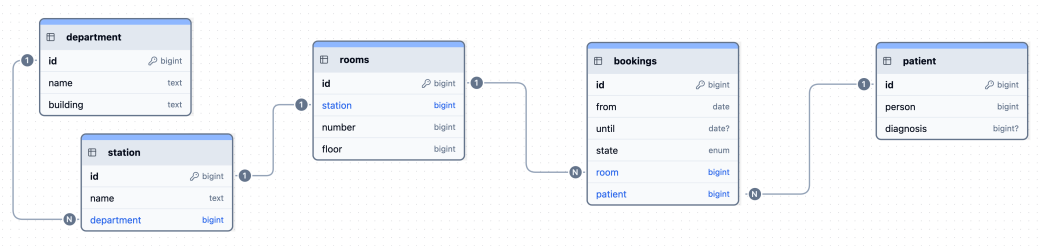

Abbildung 3: Resultierende Klassen zur Patienten- und Raumverwaltung

Die meisten Attribute der Entitäten sind selbsterklärend. Es sei jedoch darauf hingewiesen, dass das Attribut `until` der Entität `Booking` ein Optional darstellt, da das Ende der Behandlung häufig erst nach mehreren Untersuchungen prognostiziert werden kann und dementsprechend nachgetragen werden muss.

Enum `booking_state`

Der Enum `booking_state` spiegelt den Lebenszyklus einer Buchung sowie den aktuellen Status eines Patientenaufenthalts im System wider. Er dient als Grundlage für die Prozesssteuerung von der initialen Terminierung bis hin zum Anbschluss oder einer Stornierung der Behandlung.

Status	**Beschreibung**
pending	Ausstehend; die Buchungsanfrage ist eingegangen, wurde aber noch nicht final bestätigt.
confirmed	Bestätigt; der Termin wurde geprüft und ist fest im System eingeplant.
checked_in	Eingecheckt; der Patient ist im Krankenhaus erschienen und die Aufnahme wurde vollzogen.
completed	Abgeschlossen; die Behandlung oder der Aufenthalt wurde regulär beendet.
cancelled	Storniert; der Termin wurde entweder durch das Personal oder den Patienten abgesagt.
no_show	Nicht erschienen; der Patient hat den Termin ohne vorherige Absage versäumt.
checked_out_early	Vorzeitig beendet; der Patient hat das Krankenhaus vor dem geplanten Behandlungsende verlassen (z. B. Entlassung gegen ärztlichen Rat).

Tabelle 9: Zustände des Buchungsprozesses (Booking States)

Die Abbildung des Buchungsstatus über einen fest definierten Enum-Typ gewährleistet eine konsistente Prozesslogik innerhalb des Systems. Dies verhindert inkonsistente Zustandsübergänge in der Datenbank und ermöglicht eine präzise statistische Auswertung der Bettenbelegung sowie der Ausfallraten (No-Shows).

Zusätzlich sorgt die strikte Typisierung dafür, dass die Geschäftslogik der Anwendung jederzeit auf validen Daten operiert, was die Fehleranfälligkeit bei der Statusverwaltung erheblich reduziert.

7 Finales Klassendiagramm

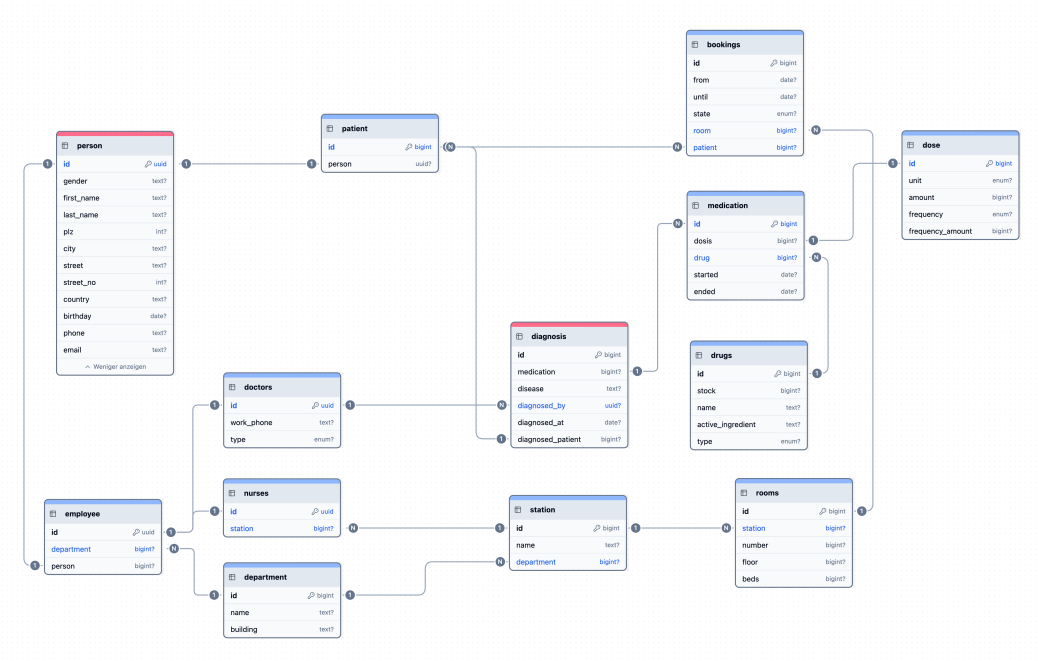

Abbildung 4: Hospitalmanagement

8 KI Nutzung

Die Erstellung dieser Dokumentation wurde durch den Einsatz von KI-Modellen unterstützt, die vorrangig die Rolle eines digitalen Reviewers übernahmen. Das betrifft einerseits die sprachliche Professionalisierung eigener Textentwürfe und andererseits die Bereitstellung der LaTeX-Template-Struktur. Auf technischer Ebene wurden alle Klassen unabhängig entwickelt, die KI kam hier ausschließlich punktuell zur Validierung der Logik zum Einsatz. Damit konnte zwar der Prozess der Korrektur und Formatierung optimiert werden, die inhaltliche Grundlage sowie alle maßgeblichen Architektur- und Designentscheidungen bleiben jedoch das Ergebnis einer eigenständigen Leistung des Teams.

Literatur

[1] Bundesamt für Sicherheit in der Informationstechnik (BSI). *Kritische Infrastrukturen (KRITIS)*. 2026. URL: https://www.bsi.bund.de/DE/Themen/Regulierte-Wirtschaft/Kritische-Infrastrukturen/kritis_node.html (besucht am 26. 03. 2026).

[2] DarkKnight25. *Medical Medicine Dataset*. URL: https://huggingface.co/datasets/darkknight25/medical_medicine_dataset/blob/main/medical_medicine_dataset.jsonl (besucht am 26. 03. 2026).

[3] Mendeley Data. *Dataset on Drug-Drug Interactions*. URL: https://data.mendeley.com/datasets/2cxccsxydc/1 (besucht am 26. 03. 2026).

[4] Docker, Inc. *Docker – Accelerated Container Application Development*. 2026. URL: https://www.docker.com/ (besucht am 13. 04. 2026).

[5] JetBrains. *DataGrip – The Cross-Platform IDE for Databases & SQL*. 2026. URL: https://www.jetbrains.com/datagrip/ (besucht am 13. 04. 2026).

[6] NDR. *Daten von LUP-Kliniken im Darknet veröffentlicht*. 2024. URL: https://www.ndr.de/nachrichten/mecklenburg-vorpommern/Daten-von-LUP-Kliniken-im-Darknet-veroeffentlicht,lupkliniken114.html (besucht am 26. 03. 2026).

[7] Swain Project. *Synthetic Data Person*. URL: https://www.kaggle.com/datasets/swainproject/synthetic-data-person (besucht am 26. 03. 2026).

[8] The pgAdmin Development Team. *pgAdmin – PostgreSQL Tools*. 2026. URL: https://www.pgadmin.org/ (besucht am 13. 04. 2026).